



THEKNIFE

MANUALE TECNICO

BENNAJIM ALI, BARBOTTI FEDERICO, OITTIJO AHMED SARKAR

LABORATORIO INTERDISCIPLINARE A, UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

9/02/2026 – V1.0

SOMMARIO

1. Introduzione.....	4
2. Librerie esterne utilizzate	4
3. Struttura generale del sistema di classi	4
3.1. Classi per il funzionamento dell'applicazione	4
3.2. Classi per interfaccia grafica e animazioni	5
4. Classi principali	5
4.1. Bootstrap e Core dell'applicazione	5
4.1.1. Launcher.java	5
4.1.2. App.java	5
4.2. Configurazione e iniezione delle Dipendenze	6
4.2.1. DependencyContainer.java	6
4.2.2. AppConfig.java	7
4.3. Gestione del Filesystem e delle Risorse	7
4.3.1. FileProvider.java (Interfaccia)	7
4.3.2. DefaultFileProvider.java	7
4.3.3. ResourceFileHelper.java	8
4.4. Strato di Accesso ai Dati (Data Access Layer)	8
4.4.1. CSVManager.java	8
4.5. Modelli di Dominio (Model Layer)	9
4.5.1. User.java	9
4.5.2. Restaurant.java	9
4.5.3. Review.java	10
4.5.4. FavoriteRestaurant.java	10
4.6. Strato di Servizio (Business Logic Layer)	10
4.6.1. Interfacce del Servizio	11
4.6.2. Implementazioni del Servizio	11
4.7. Gestione della Sessione Utente	13
4.7.1. SessionManager.java	14

4.8.	Strato di Presentazione (ViewModel Layer - MVVM)	14
4.8.1.	RestaurantListViewModel.java	14
4.8.2.	RestaurantDetailsViewModel.java	15
4.8.3.	ReviewViewModel.java	15
4.9.	Componenti di Utilità (Utility Layer).....	16
4.9.1.	Logger.java.....	16
4.9.2.	PasswordHasher.java	16
4.9.3.	GeoValidator.java.....	16
4.9.4.	CuisineImageManager.java	17
5.	Scelte algoritmiche e di strutture dati	17
5.1.	Strutture Dati	17
5.1.1.	Cache Dati in Memoria: java.util.List	17
5.1.2.	Binding UI: javafx.collections.ObservableList	18
5.1.3.	Service Locator: java.util.Map.....	18
5.2.	Algoritmi.....	19
5.2.1.	Algoritmo di Hashing delle Password (PBKDF2)	19
5.2.2.	Algoritmo di Ricerca Testuale	19
5.2.3.	Algoritmo di Mapping basato su Parole Chiave.....	20
6.	Gestione dei file	21
6.1.	File di Dati e Loro Funzione	21
6.2.	Formattazione dei File.....	21
6.3.	La Cartella resources e il Processo di Copia.....	21
6.4.	Utility per l'Aggiunta di ID	21
6.5.	Modalità di Accesso ai File	22
7.	Bibliografia	23

1.Introduzione

TheKnife è una applicazione sviluppata nell'ambito del progetto di Laboratorio A per il corso di laurea in Informatica dell'Università degli Studi dell'Insubria.

Il progetto è sviluppato in java jdk 24 e funziona con java 21+, usa un'interfaccia grafica costruita con JavaFX 21.0.4.

2.Librerie esterne utilizzate

Per lo sviluppo di questa applicazione sono state utilizzate alcune librerie esterne:

- **OpenJFX / JavaFx 21.0.4:** usata per lo sviluppo di tutta l'interfaccia grafica
- **Apache Common CSV 1.11.0:** è usata per la persistenza su file CSV.
- **Junit Jupiter 5.9.1:** è un framework per eseguire i test.
- **Gradle OpenJFX Plugin 0.1.0:** è un plugin gradle che semplifica la configurazione e packaging di JavaFX

3.Struttura generale del sistema di classi

Il progetto è strutturato in 2 rami: funzionamento dell'applicazione, interfaccia grafica e animazioni.

3.1. Classi per il funzionamento dell'applicazione

- Launcher
- App
- Dependency Container
- AppConfig
- FileProvider
- DefaultFileProvider
- resourceFileHelper
- SessionManager
- CSVManager
- User
- Restaurant
- Review
- FavoriteRestaurant
- IUserService
- IRestaurantService
- IReviewService
- IFavoriteService
- UserService
- RestaurantService
- ReviewService
- FavoriteService
- RestaurantListViewModel

- RestaurantDetailsViewModel
- ReviewViewModel
- Logger
- PasswordHasher
- GeoValidator
- CuisinedImageManager
- TestCSVIdAdder

3.2. Classi per interfaccia grafica e animazioni

Le classi utilizzate per l'interfaccia grafica e le animazioni sono: LoginScreen, RegisterScreen, RestaurantListView, RestaurantDetailView, RestaurantFormView, FavoriteRestaurantsView, MyRestaurantsView, UserProfileView, ReviewView, ModalManager, ModalStackPane, AnimationUtils.

Verranno presentate ora, nel dettaglio, solo le classi per il funzionamento dell'applicazione, non verrà affrontata invece una discussione delle classi per l'interfaccia grafica perché non espressamente richiesta nelle specifiche del progetto.

4. Classi principali

In questo paragrafo vengono analizzate le classi fondamentali per il funzionamento dell'applicazione "The Knife". L'architettura di questa piattaforma adotta un approccio multi-layer per garantire una chiara separazione delle responsabilità, manutenibilità e testabilità. I layer principali includono il Presentation layer (che implementa il pattern MVVM), il Business Logic Layer (Service Layer), il Data Access Layer e un Utiliy Layer trasversale. Questa spiegazione seguirà un approccio "top-down", partendo dal punto di ingresso dell'applicazione e scendendo progressivamente nei vari layer architetturali fino ai componenti di più basso livello.

4.1. Bootstrap e Core dell'applicazione

Il ciclo di vita dell'applicazione inizia con il processo di bootstrap, gestito da due classi distinte che separano il punto di ingresso nativo dalla logica dell'applicazione JavaFX.

4.1.1. Launcher.java

- **Responsabilità:** funge da punto di ingresso primario (main method) per l'applicazione java. Invoca il metodo ***main*** della classe ***App***.
- **Funzionamento:** L'unica responsabilità di questa classe è avviare il runtime di JavaFX chiamando il metodo `launch()` della classe "App". La separazione tra launcher e App è una pratica consolidata in JavaFX, implementata per risolvere potenziali problemi di caricamento delle librerie modulari di JavaFX in diversi ambienti di esecuzione.

4.1.2. App.java

- **Responsabilità:** è la classe principale dell'applicazione, responsabile della gestione del ciclo di vita dell'interfaccia grafica (UI) e dell'orchestrazione dell'inizializzazione dei componenti core.

- **Eredità:** estende `javafx.application.Application`.
- **Funzionamento:**
 - Il metodo **`start(Stage(primaryStage))`** è il punto di ingresso per tutta la logica dell'applicazione.
 - Inizializzazione: All'avvio, il metodo **`start`** si occupa di inizializzare il `DependencyContainer`, garantendo che tutti i servizi e i gestori di dati siano pronti per essere utilizzati.
 - Gestione della scena: Imposta e visualizza la scena iniziale (in questo caso, **`HomeScene`**) all'interno del **`primaryStage`**.
L'architettura dell'applicazione è single-stage, il che significa che un'unica finestra principale (**`Stage`**) viene utilizzata per l'intera durata dell'applicazione. Le diverse viste vengono caricate e scambiate dinamicamente all'interno di questa finestra, offrendo un'esperienza utente fluida e coesa.

4.2. Configurazione e iniezione delle Dipendenze

Il cuore dell'architettura disaccoppiata risiede nel meccanismo di Iniezione Dipendenze (DI), implementato tramite il pattern Service Locator. Questo centralizza la creazione e la fornitura delle dipendenze in tutta l'applicazione.

4.2.1. `DependencyContainer.java`

- **Pattern architetturale:** È un **`IoC Container`** leggero basato su **`Service Locator`**. Gestisce l'inversione del controllo centralizzando la risoluzione delle dipendenze, supportando sia l'istanziamento immediato (Eager) che differito (Lazy).
- **Responsabilità:** La classe incapsula tre responsabilità ortogonali:
 - **Service Registry:** Mappa interfacce a implementazioni concrete (`Map<Class, Object>`).
 - **Lifecycle Management:** Gestisce gli scope, primariamente Singleton (istanza unica) e Lazy Singleton (tramite Supplier e memoization al primo accesso).
 - **Dependency Resolution:** Risolve il grafo delle dipendenze a runtime, fornendo l'istanza corretta richiesta dal client.
- **Il suo Ruolo:** Agisce come **Composition Root** dell'applicazione. È l'unico punto nel sistema dove le dipendenze concrete sono conosciute, istanziate e "cablate" (wired) insieme. Questo previene il **Service Control Anti-pattern**, dove le classi di basso livello controllano l'istanziamento delle proprie dipendenze.
- **Vantaggi del Design (SOLID Principles):**
 - **DIP (Dependency Inversion Principle):** I client dipendono da astrazioni, non da classi concrete.
 - **Decoupling:** Massima separazione tra logica di creazione e logica di utilizzo.

- **Manutenibilità:** Separa nettamente la logica di creazione degli oggetti dalla logica di utilizzo (business logic).
- **Centralizzazione:** Il metodo `initializeAllServices` agisce come **Configuration Object** atomico. Garantisce consistenza configurando tutte le dipendenze in un unico blocco transazionale all'avvio.
- **Testabilità:** Facilita il Mocking. Essendo basato su interfacce, permette di iniettare facilmente Mock o Stub durante i test (es. **`MockUserService`**) e di resettare lo stato globale con **`clear()`**, garantendo l'isolamento dei test.
- **Funzionamento:** Utilizza Generics per la Type Safety (**`get(Class<T>)`**).
 - **O(1) Lookup:** recupero istantaneo tramite `HashMap`.
 - **Lazy Loading:** Ottimizza l'avvio differendo la creazione di oggetti pesanti al primo utilizzo reale tramite **`Supplier`**.

4.2.2. AppConfig.java

- **Responsabilità:** Agisce come un contenitore globale per costanti di configurazione statiche e hardcoded, utilizzate principalmente per scopi di sviluppo e test.
- **Funzionamento:** È una classe **`final`** con un costruttore privato, che espone una serie di costanti **`public static final`**. Queste includono un flag **`DEBUG`** per abilitare o disabilitare funzionalità di sviluppo come la compilazione automatica e le credenziali (email/password) per utenti di test predefiniti. I valori sono impostati direttamente nel codice sorgente all'interno di un blocco statico.
- **Scopo:** Il suo unico scopo è fornire un accesso centralizzato e semplice a valori di configurazione fissi che sono condivisi all'interno del team di sviluppo e non sono destinati a essere modificati dagli utenti finali.

4.3. Gestione del Filesystem e delle Risorse

Per garantire la portabilità e la testabilità, l'accesso al filesystem è completamente astratto. L'applicazione gestisce inoltre in modo robusto le risorse interne (dati di default) che devono essere rese disponibili sul filesystem dell'utente al primo avvio.

4.3.1. FileProvider.java (Interfaccia)

- **Responsabilità:** Definisce un contratto per l'astrazione dell'accesso al filesystem. Specifica un metodo (**`getCSVPath`**) che i componenti possono utilizzare per ottenere l'oggetto `Path` (Java NIO) di un file o dati a partire da un nome logico.
- **Vantaggi del Design:**
 - **Portabilità:** Disaccoppia la logica di business dall'implementazione specifica del filesystem.
 - **Testabilità:** Permette di fornire implementazioni alternative (es. un **`MockFileProvider`** che lavora con un filesystem in-memory) durante i test, eliminando la dipendenza dal disco fisico.

4.3.2. DefaultFileProvider.java

- **Responsabilità:** È l'implementazione standard dell'interfaccia **FileProvider**.
- **Funzionamento:**
 - Mantiene una mappa interna che associa i nomi logici dei file (es. "users.csv") ai loro percorsi fisici (**java.nio.file.Path**).
 - Nel suo costruttore, collabora con **ResourceFileHelper** per inizializzare l'ambiente di lavoro. Per ogni file di dati, controlla se esiste già nella directory dell'utente. In caso contrario, lo copia dalle risorse interne del JAR.
 - Il metodo **getCsvPath** consulta la mappa per restituire il percorso effettivo del file richiesto.

4.3.3. ResourceFileHelper.java

- **Responsabilità:** Gestisce il processo di *bootstrap* delle risorse dell'applicazione al primo avvio.
- **Funzionamento:**
 - Fornisce metodi per estrarre file CSV di dati dal pacchetto JAR dell'applicazione e salvarli in una directory sul filesystem locale dell'utente (.theknife/data) in base al sistema operativo ospite.
 - Questo processo è cruciale perché i file all'interno di un JAR sono di sola lettura. Copiandoli all'esterno, si garantisce che l'applicazione possa leggere e scrivere sui propri file di dati.
 - L'operazione di copia viene eseguita solo se il file di destinazione non esiste già, evitando di sovrascrivere i dati dell'utente ad ogni avvio.

4.4. Strato di Accesso ai Dati (Data Access Layer)

La persistenza dei dati è gestita tramite un Data Access Layer (DAL) il cui componente centrale è un repository generico per la gestione di file CSV.

4.4.1. CSVManager.java

- **Responsabilità:** Componente chiave del Data Access Layer. Fornisce un'interfaccia generica e riutilizzabile per le operazioni CRUD (*Create, Read, Update, Delete*) su file CSV.
- **Design:** È una classe generica (**`CSVManager<T>`**), il che la rende agnostica rispetto al tipo di dato (modello) che deve persistere. Utilizza alcuni principi dei seguenti pattern: Data Access Object (DAO) e Generic Repository.
- **Funzionamento:**

- **Caching in Memoria:** Al momento della sua creazione, **CSVManager** carica l'intero contenuto del file CSV in una `List<T>` in memoria. Tutte le operazioni di lettura successive (ricerche, filtri) avvengono su questa cache, garantendo prestazioni elevate ed evitando accessi ripetuti al disco.
- **Parsing e Serializzazione Funzionale:** Per disaccoppiare la logica di gestione del CSV dalla struttura specifica dei modelli, il costruttore accetta due Function come argomenti:
 - `Function<String, T> parser`: una funzione che trasforma una riga CSV (stringa) in un oggetto di tipo T.
 - `Function<T, String> serializer`: una funzione che trasforma un oggetto di tipo T in una riga CSV.
- **Thread Safety:** Le operazioni che modificano la cache e il file (**save(T object)**, **replace(Function<T, Boolean> matcher, T newValue)**, **removeIf(Function<T, Boolean> predicate)**) sono synchronized per garantire l'atomicità e prevenire *race condition*.
- **Persistenza differita:** Le operazioni di modifica (**save**, **replace**, **removeIf**) aggiornano la cache in memoria. La scrittura fisica su disco avviene solo quando viene invocato esplicitamente il metodo **saveToDisk()**, permettendo di raggruppare più modifiche in un'unica operazione di I/O (Batch Writing) per efficienza."

4.5. Modelli di Dominio (Model Layer)

I modelli di dominio sono **POJO** (*Plain Old Java Object*) che rappresentano le entità di business del sistema. Sono strutture dati che incapsulano anche la logica di (de)serializzazione per il formato CSV.

I metodi di queste classi dominio `fromCsv` e `toCsv` servono per la serializzazione.

4.5.1. User.java

- **Responsabilità e Pattern:** Rappresenta l'entità Utente nel sistema. È progettata come Oggetto Immutabile (classe e attributi ***final***, nessun setter) per garantire l'integrità dei dati e la sicurezza in ambienti concorrenti (thread-safety). Le modifiche allo stato avvengono creando nuove istanze (es. **withPassword**).
- **Attributi Gestiti:**
 - Identificativo: **email** (funge da chiave univoca logica).
 - Anagrafica: **name**, **surname**, **dateOfBirth**.
 - Sicurezza e Ruoli: **password** (hash memorizzato) e **role** (definisce i permessi nel sistema).
 - Geolocalizzazione: **latitude**, **longitude**.

4.5.2. Restaurant.java

- **Responsabilità e Pattern:** Rappresenta l'entità Ristorante nel dominio applicativo. È progettata come Oggetto Immutabile (classe e attributi final) per garantire la coerenza dei dati. Utilizza tecniche di ottimizzazione della memoria (String Interning) per ridurre l'occupazione RAM su campi ripetitivi (es. città, cucina).
- **Attributi Gestiti:**
 - Identificativi: id (Chiave Primaria).
 - Localizzazione: address , location (Città), latitude , longitude e distanceKm (calcolato a runtime).
 - Dettagli Offerta: price (Fascia prezzo), cuisine (Tipo cucina), award (Premi/Michelin), greenStar e facilitiesAndServices.
 - Contatti e Info: phoneNumber , url , websiteUrl e description.
 - Proprietà: restaurateurEmail (Chiave esterna verso l'utente proprietario).

4.5.3. Review.java

- **Responsabilità e Pattern:** Rappresenta una recensione rilasciata da un utente per un ristorante. È progettata come **Oggetto Immutabile** (classe e attributi final, nessun setter) per garantire la thread-safety e la consistenza. Le modifiche (es. aggiunta di una risposta) generano nuove istanze tramite metodi with... (pattern Wither).
- **Attributi Gestiti:**
 - Identificativi: id (Univoco recensione)
 - Relazioni: restaurantId (Riferimento al ristorante) e userEmail (Riferimento all'autore).
 - Contenuto: rating (Voto 1-5 stelle), comment (Testo) e reviewDate (Data pubblicazione).
 - Interazione: restaurateurResponse (Risposta del proprietario) e clientResponse (Contro-risposta del cliente).
 - Stato: isVerified (Flag di verifica).

4.5.4. FavoriteRestaurant.java

- **Responsabilità e Pattern:** Modello di associazione che implementa la relazione molti-a-molti tra User e Restaurant. Rappresenta il concetto di "Preferito" nel sistema. È progettata come **Oggetto Immutabile** (final) per garantire la consistenza delle relazioni salvate.
- **Attributi Gestiti:**
 - Chiave Primaria Composita: userEmail (Chi ha messo il like) e restaurantId (A quale ristorante).
 - Compatibilità Legacy: userName e restaurantName (Campi mantenuti per supportare vecchie versioni dei dati CSV prima dell'introduzione degli ID numerici).

4.6. Strato di Servizio (Business Logic Layer)

Il Service Layer implementa la logica di business dell'applicazione. È progettato separando rigorosamente le interfacce (i contratti) dalle implementazioni concrete.

4.6.1. Interfacce del Servizio

4.6.1.1. IUserService.java

- **Responsabilità:** Definisce il contratto per tutte le operazioni di business relative alla gestione del ciclo di vita e dell'autenticazione degli utenti.
- **Contratto:** Specifica i metodi per la verifica delle credenziali (*validateCredentials*), il salvataggio o registrazione di nuovi utenti (*saveUser*), la verifica di esistenza (*emailExists*) e il recupero del profilo tramite identificativo univoco (*findUserByEmail*) o nome legacy (*findUserByName*).

4.6.1.2. IRestaurantService.java

- **Responsabilità:** Definisce il contratto per le operazioni di Business Logic relative all'accesso, ricerca e gestione dei ristoranti, disaccoppiando la presentazione della persistenza.
- **Contratto:** Specifica i metodi per la ricerca paginata e filtrata (*searchRestaurantsRange*), il recupero puntuale tramite ID (*findRestaurantById*) o nome (*findRestaurantByName*), l'inserimento di nuovi ristoranti (*addRestaurant*), la ricerca geospaziale per prossimità (*findNearby*) e il recupero per proprietario (*getRestaurantsByRestaurateur*).

4.6.1.3. IReviewService.java

- **Responsabilità:** Definisce il contratto per la gestione completa delle recensioni, incluse le interazioni tra utenti e ristoratori e l'analisi dei dati.
- **Contratto:** Specifica i metodi per la gestione CRUD delle recensioni (*addReview*, *updateReview*, *deleteReview*), il recupero filtrato (*getReviewsForRestaurant*, *getReviewsByUser*, *getRecentReviews*), il calcolo di metriche (*getAverageRating*, *getRatingDistribution*) e la gestione delle risposte (*addRestaurateurResponse*, *addClientResponse*).

4.6.1.4. IFavoriteService.java

- **Responsabilità:** Definisce il contratto per la gestione della relazione molti-a-molti tra utenti e ristoranti preferiti.
- **Contratto:** Gestisce l'aggiunta/rimozione (*addFavorite/removeFavorite*), il recupero della lista completa (*getUserFavorites* o *getUserFavoriteRestaurants*) e la verifica dello stato di preferenza (*isFavorite*).

4.6.2. Implementazioni del Servizio

4.6.2.1. UserService.java

- **Responsabilità:** Implementa **UserService** gestendo il ciclo di vita e la persistenza degli utenti su file CSV.
- **Collaborazioni:** Interagisce con `CSVManager<User>` per la persistenza dei dati e con **PasswordHasher** per la verifica sicura delle credenziali.
- **Logica di Business:**
 - **Autenticazione:** Valida le credenziali utente verificando email e hash della password in modo sicuro.
 - **Gestione CRUD:** Gestisce registrazione e aggiornamento profili con validazione dei dati (email, coordinate), hashing automatico delle password e controllo duplicati.
 - **Lookup Ottimizzato:** Utilizza indici in memoria per ricerche rapide ($O(1)$) tramite email.
 - **Retrocompatibilità:** Include meccanismi di migrazione automatica per supportare e convertire dati utente salvati in formati legacy.

4.6.2.2. RestaurantService.java

- **Responsabilità:** Implementa **RestaurantService** per l'accesso ai dati dei ristoranti.
- **Collaborazioni:** Utilizza `CSVManager<Restaurant>` per la persistenza.
- **Logica di Business:**
 - **Gestione Dati Eager Loading:** Carica l'intero dataset dei ristoranti in memoria all'avvio per garantire accessi e ricerche in tempo reale.
 - **Indicizzazione e Ricerca:** Implementa indici in memoria (Map) per lookup $O(1)$ per ID ed Email, e algoritmi di ricerca full-text con filtri su più campi (nome, cucina, località) e paginazione dei risultati.
 - **Analisi Geospaziale:** Include funzionalità di ricerca per prossimità (`findNearby`) utilizzando la formula dell'Haversine per calcolare le distanze.
 - **Gestione CRUD e Persistenza:** Gestisce la creazione di nuovi ristoranti con generazione automatica di ID univoci sequenziali e persistenza sincronizzata su file CSV, garantendo l'integrità dei dati tramite validazione e retrocompatibilità dei formati

4.6.2.3. ReviewService.java

- **Responsabilità:** Implementa **ReviewService** per la gestione delle recensioni.

- **Collaborazioni:** Utilizza `CSVManager<Review>` e *RestaurantService.java*.
- **Logica di Business:**
 - **Gestione CRUD Avanzata:** Gestisce l'intero ciclo di vita delle recensioni (creazione, lettura, aggiornamento, cancellazione) con persistenza su CSV e verifica rigorosa dell'ownership per modifiche e risposte (ristoratore/cliente).
 - **Caching e Indicizzazione:** Carica le recensioni in memoria e mantiene indici ottimizzati per ricerche rapide ($O(1)$) raggruppate per ID, ristorante e utente, evitando accessi disco ripetuti.
 - **Aggregazione Dati:** Calcola in tempo reale metriche statistiche sui ristoranti, inclusi media voti, conteggio totale e distribuzione dei rating (istogramma stelle), direttamente in memoria.
 - **Immutabilità e Concorrenza:** Utilizza pattern di immutabilità (Wither) per gli aggiornamenti (es. aggiunta risposte) e sincronizza le operazioni di scrittura per garantire thread-safety e consistenza dei dati.

4.6.2.4. FavoriteService.java

- **Responsabilità:** Implementa *IFavoriteService*, gestendo la relazione molti-a-molti tra utenti e ristoranti.
- **Collaborazioni:** Utilizza `CSVManager<FavoriteRestaurant>` per gestire il salvataggio su file e il caricamento in memoria.
- **Logica di Business:**
 - **Gestione Relazionale Semplificata:** Gestisce le associazioni molti-a-molti tra Utenti e Ristoranti (Preferiti) persistendo le coppie univoche su file CSV.
 - **Doppia Indicizzazione in Memoria:** Mantiene due HashMap parallele (favoritesByUser e favoritesByRestaurant) per garantire lookup bidirezionali istantanei ($O(1)$) senza scansionare l'intero dataset.
 - **Prevenzione Duplicati:** Implementa controlli preventivi (isFavorite) durante l'inserimento per evitare ridondanze sia logiche che fisiche nel file di persistenza.
 - **Sincronizzazione Memoria-Disco:** Coordina le operazioni di scrittura (addFavorite , removeFavorite) aggiornando atomicamente sia le strutture dati in cache (liste e mappe) che il file su disco per garantire consistenza.

4.7. Gestione della Sessione Utente

Lo stato di autenticazione globale è gestito da un Singleton dedicato per garantire un punto di accesso unico e coerente in tutta l'applicazione.

4.7.1. SessionManager.java

- **Pattern Architetturale:** Singleton.
- **Responsabilità:** Mantiene lo stato di autenticazione globale per l'intera applicazione. È l'unica "fonte di verità" per sapere quale utente è attualmente loggato.
- **Funzionamento:**
 - **Singleton:** Un'unica istanza di SessionManager viene creata e condivisa in tutta l'applicazione, accessibile tramite un metodo statico getInstance().
 - **Gestione della Sessione:** Fornisce metodi per:
 - setCurrentUser(User user): imposta l'utente corrente (effettua il login)
 - logout(): Rimuove l'utente memorizzato, terminando la sessione.
 - getCurrentUser(): Restituisce l'utente attualmente loggato (o null se nessuno è loggato).
 - isLoggedIn(): Un metodo di convenienza per controllare rapidamente se esiste una sessione attiva.

4.8. Strato di Presentazione (ViewModel Layer - MVVM)

Il Presentation Layer è basato sul pattern MVVM (Model-View-ViewModel) per disaccoppiare la logica della UI (View) dalla logica di presentazione e dallo stato (ViewModel). Questo layer fa ampio uso delle proprietà e del data binding di JavaFX.

4.8.1. RestaurantListViewModel.java

- **Responsabilità:** Fornisce dati e logica di presentazione per la schermata che visualizza l'elenco dei ristoranti (RestaurantListView).
- **Collaborazioni:** Dipende da IRestaurantService per recuperare i dati dei ristoranti e SessionContext.
- **Funzionamento (MVVM):**
 - **Mantenimento dello Stato:** Contiene lo stato della vista, in particolare la lista dei ristoranti da visualizzare (ObservableList).
 - **Proprietà Osservabili:** Espone i dati della UI tramite le *Properties* di JavaFX (es. ObservableList<Restaurant>, StringProperty). La View si "binda" (collega) a queste proprietà.
 - **Data Binding:** Grazie al binding, quando la lista di ristoranti nel ViewModel viene modificata (ad esempio, a seguito di una ricerca), la UI si aggiorna automaticamente, senza che sia necessario scrivere codice manuale per manipolare la UI.

- **Gestione Azioni Utente:** Espone metodi pubblici (es. *performSearch(String query)*, *refresh()*) che vengono invocati dalla View in risposta a eventi dell'utente (es. la pressione di un tasto in una casella di ricerca). Questi metodi contengono la logica per chiamare il *RestaurantService* e aggiornare la *ObservableList* dei ristoranti, innescando l'aggiornamento automatico della UI.

4.8.2. RestaurantDetailsViewModel.java

- **Responsabilità:** Fornisce dati e logica di presentazione per la schermata di dettaglio di un singolo ristorante.
- **Collaborazioni:** Dipende da *IRestaurantService*, *IReviewService*, *SessionContext*.
- **Funzionamento (MVVM):**
 - **Mantenimento dello Stato:** Mantiene lo stato completo della vista, inclusi i dettagli del ristorante selezionato, la lista delle sue recensioni.
 - **Proprietà Osservabili:** Espone i dettagli del ristorante tramite singole *StringProperty* (es. *restaurantName* , *restaurantAddress*) e le recensioni tramite una *ObservableList<Review>* .
 - **Logica di Business Delegata:** Carica i dettagli del ristorante e le relative recensioni invocando i servizi appropriati. Calcola statistiche aggregate come la media dei voti e il conteggio recensioni (*updateReviewStatistics*).
 - **Gestione Azioni Utente:** Espone metodi per aggiornare i dati (*refreshReviews*) e gestire le risposte dei ristoranti (*addRestaurateurResponse*). Determina se l'utente corrente può aggiungere una recensione tramite la property *canAddReview*.

4.8.3. ReviewViewModel.java

- **Responsabilità:** Gestisce la logica di presentazione e validazione per il form di creazione e modifica di una recensione.
- **Collaborazioni:** Dipende da *IReviewService* per il salvataggio dei dati e *SessionContext* per verificare l'identità dell'autore.
- **Funzionamento (MVVM):**
 - **Gestione Stato Form:** Mantiene lo stato dei campi di input (voto, commento) e lo stato della validazione (property *isValid*, *errorMessage*)
 - **Validazione Reattiva:** verifica in tempo reale la correttezza dei dati inseriti (es. lunghezza commento massima, obbligatorietà del voto) aggiornando le proprietà osservabili per feedback immediato.
 - **Gestione Operazioni:** Gestisce l'invio asincrono della recensione (*submitReview*), supportando sia la modalità "nuova recensione" (*initializeForNewReview*) che "modifica esistente" (*initializeForEdit*).

- **Proprietà osservabili:** Espone proprietà bidirezionali per i controlli di input (*rating*, *comment*) e proprietà di stato per la UI (*isSubmitting*, *successMessage*). Fornisce inoltre metodi helper per la lunghezza del commento (*getCommentLength*, *getMaxCommentLength*)

4.9. Componenti di Utilità (Utility Layer)

L'Utility Layer contiene classi trasversali che forniscono funzionalità di supporto a vari layer dell'applicazione.

4.9.1. Logger.java

- **Responsabilità:** Fornisce un meccanismo di logging leggero e standard per l'intera applicazione.
- **Funzionamento:**
 - Utilizza un metodo factory statico (*getLogger*) per creare istanze configurate per specifiche classi, permettendo di tracciare l'origine del messaggio.
 - Fornisce vari livelli di log (es. *info*, *warning*, *error*).
 - Ogni messaggio di log è formattato per includere il livello di log e il nome della classe, garantendo un output su console organizzato per il debugging.

4.9.2. PasswordHasher.java

- **Responsabilità:** Componente di sicurezza fondamentale per la gestione delle password.
- **Algoritmo:** Utilizza **PBKDF2 con HMAC-SHA256**, uno standard industriale robusto che include un salt casuale per ogni password e 100.000 iterazioni di hashing per prevenire attacchi rainbow table e brute-force.
- **Funzionamento:**
 - Espone due metodi statici principali: *hashPassword* per la registrazione e *verifyPassword* per il login.
 - L'output non è un semplice hash, ma una stringa codificata che contiene tutti i parametri necessari per la verifica futura (algoritmo, iterazioni, salt e hash finale).
 - Include una logica di retro-compatibilità per vecchi hash SHA-256 (se presenti), ma genera sempre nuovi hash con lo standard PBKDF2 più sicuro.

4.9.3. GeoValidator.java

- **Responsabilità:** Fornisce metodi di utilità per validare le coordinate geografiche (latitudine e longitudine) secondo lo standard WGS84.
- **Funzionamento:**

- - Implementa controlli di range statici e stateless (`isValidLatitude` , `isValidLongitude`) per assicurare che la latitudine sia tra -90 e +90 e la longitudine tra -180 e +180.
- Fornisce un metodo di validazione rigorosa (`validateCoordinates`) che lancia un'eccezione (`IllegalArgumentException`) se le coordinate sono invalide, utile per bloccare il salvataggio di dati corrotti.

4.9.4. CuisineImageManager.java

- **Pattern Architetturale:** Singleton.
- **Responsabilità:** Gestisce centralmente il mapping tra i tipi di cucina (stringhe testuali) e le relative risorse immagine.
- **Funzionamento:**
 - **Singleton:** Garantisce un'unica istanza per centralizzare la gestione delle risorse.
 - **Lazy Loading e Caching:** Le immagini vengono caricate dal classpath solo quando richieste per la prima volta e poi mantenute in una cache in-memory per ottimizzare le letture I/O successive.
 - **Mapping Flessibile:** Mappa diverse tipologie di cucina a una macro-categoria di immagine (es. "Sushi", "Ramen" vengono mappati a [japanese.jpg](#)). Supporta anche corrispondenze parziali (es. "Tuscan" -> [italian.jpg](#)).
 - **Fallback:** Se non viene trovata alcuna corrispondenza per una data cucina, restituisce un'immagine di default predefinita.

5. Scelte algoritmiche e di strutture dati

5.1. Strutture Dati

Le strutture dati sono state scelte per bilanciare performance, semplicità di implementazione e coerenza con il framework applicativo (JavaFX).

5.1.1. Cache Dati in Memoria: `java.util.List`

Il layer di servizio adotta un'architettura di caching ibrida che carica l'intero dataset in RAM all'avvio (Eager Loading), garantendo massima reattività all'interfaccia utente.

- **Strutture dati utilizzate:**
 - Master List (`ArrayList<T>`): Contiene l'elenco completo e ordinato di tutti i record. È la struttura primaria per iterazioni e scansioni sequenziali.
- **Funzionamento:** Le operazioni di scrittura mantengono la consistenza aggiornando atomicamente la lista in memoria, oltre a marcare il dataset come "dirty" per la successiva persistenza su disco.
- **Complessità:**

- Inserimento (Add): $O(1)$ ammortizzato (Aggiunta in coda alla Lista e inserimento in Mappa).
- Aggiornamento (Update): $O(n)$ (Richiede la scansione lineare della Lista per individuare l'indice del record da sostituire).
- Scansione/Filtro generico: $O(n)$ (Iterazione completa sulla Lista).
- **Giustificazione:** L'architettura ibrida offre tempi di risposta immediati per le operazioni di lettura più frequenti (es. login, dettagli), accettando un costo lineare $O(n)$ solo per le operazioni di modifica e ricerca complesse, un compromesso ottimale per le dimensioni del dataset gestito.

5.1.2. Binding UI: `javafx.collections.ObservableList`

Per implementare il pattern MVVM in modo reattivo, i ViewModel espongono i dati alla View attraverso collezioni speciali che notificano automaticamente i cambiamenti.

- **Struttura:** `javafx.collections.ObservableList<T>` (implementata concretamente tramite `FXCollections.observableArrayList`).
- **Utilizzo:**
 - I ViewModel (es. `RestaurantListViewModel`) mantengono una `ObservableList<Restaurant>` che rappresenta lo stato corrente della visualizzazione.
 - La View (es. `RestaurantListView`) si collega a questa lista tramite Data Binding.
 - Qualsiasi modifica logica alla lista (es. applicazione di filtri, nuovi risultati di ricerca) innesca automaticamente l'aggiornamento dei componenti grafici corrispondenti (`ListView`, `TableView`), senza che il programmatore debba manipolare manualmente i nodi della UI..
- **Giustificazione:** Questa è la struttura standard dell'ecosistema JavaFX per disaccoppiare la logica di presentazione (ViewModel) dalla rappresentazione grafica (View), garantendo un'interfaccia utente fluida e sincronizzata con lo stato dei dati..

5.1.3. Service Locator: `java.util.Map`

Il `DependencyContainer` utilizza una mappa per implementare il pattern Service Locator.

- **Strutture dati:**
 - `java.util.HashMap<Class<?>, Object>` per le istanze singleton
 - `java.util.HashMap<Class<?>, Supplier<?>>` per le Factory
- **Utilizzo:**
 - La mappa associa un'interfaccia (es. `IUserService.class`) alla sua implementazione concreta (es. istanza di `UserService`).
 - All'avvio (Launcher), il container viene popolato con i servizi core.
 - Quando un componente (es. ViewModel) richiede una dipendenza, il container esegue una lookup: se l'istanza

esiste nella mappa singleton la restituisce, altrimenti la crea tramite la factory registrata.

- **Complessità:**
 - Registrazione (put): $O(1)$.
 - Risoluzione Dipendenza (get): $O(1)$ (Accesso diretto alla HashMap tramite hash della classe).
- **Giustificazione:** La HashMap garantisce tempi di risoluzione costanti e immediati indipendentemente dal numero di servizi registrati, rendendo il meccanismo di iniezione estremamente performante e scalabile.

5.2. Algoritmi

Gli algoritmi implementati privilegiano la semplicità e la leggibilità, data la scala non critica delle performance.

5.2.1. Algoritmo di Hashing delle Password (PBKDF2)

- **Algoritmo:** PBKDF2 (Password-Based Key Derivation Function 2) con HMAC-SHA256 (Standard NIST SP 800-132).
- **Descrizione:** PBKDF2 è uno standard crittografico (definito nella RFC 2898 e raccomandato dal NIST SP 800-132) progettato specificamente per la memorizzazione sicura delle password. A differenza delle semplici funzioni di hash veloci (come MD5 o SHA-1), PBKDF2 applica una funzione pseudo-casuale (in questo caso HMAC-SHA256) ripetutamente alla password di input insieme a un valore univoco casuale (Salt), rendendo computazionalmente costoso e lento il processo di derivazione della chiave.
- **Processo:**
 - Generazione di un Salt casuale univoco di 128 bit (SecureRandom) per ogni utente.
 - Esecuzione di 100.000 iterazioni di hashing HMAC-SHA256 combinando password e Salt.
 - Memorizzazione del risultato nel formato: PBKDF2\$100000\$Salt\$Hash .
 - Verifica tramite confronto a tempo costante (slowEquals) per prevenire timing attacks.
- **Giustificazione:** PBKDF2 è lo standard di settore (OWASP) per la conservazione sicura delle credenziali. L'uso del Salt impedisce l'uso di tabelle precalcolate, mentre l'elevato costo computazionale (iterazioni) mitiga l'efficacia degli attacchi paralleli su GPU in caso di data breach.

5.2.2. Algoritmo di Ricerca Testuale

- **Algoritmo:** Linear Scan con Filtering Funzionale (Java Stream API) implementato nella classe RestaurantService.java.
- **Descrizione:** Il sistema utilizza un approccio di scansione sequenziale sull'intero dataset caricato in memoria (In-Memory Database). L'algoritmo applica un predicato di filtro composito che verifica la presenza del termine di ricerca all'interno di molteplici campi dell'oggetto Restaurant

- (nome, cucina, indirizzo, località), ignorando la distinzione tra maiuscole e minuscole (case-insensitive).
- **Processo:** L'algoritmo esegue un'iterazione completa sulla lista dei ristoranti pre-caricata in RAM. Utilizza le Java Stream API per filtrare gli oggetti in base a un predicato che verifica la presenza della stringa di ricerca (normalizzata e case-insensitive) all'interno di più campi: Nome , Tipologia di Cucina , Indirizzo o Località . I risultati ottenuti vengono infine tagliati (subList) per gestire la paginazione richiesta dalla vista.
 - **Complessità:**
 - Tempo: $O(n)$, poiché nel caso peggiore deve analizzare ogni singolo ristorante nella lista.
 - Spazio: $O(k)$, dove k è il numero di risultati che soddisfano la ricerca.
 - **Giustificazione:** Poiché l'intero dataset risiede in memoria (RAM), l'accesso ai dati è estremamente veloce (nell'ordine dei nanosecondi). Per la scala dell'applicazione (fino a ~100.000 record), una ricerca lineare è impercettibile per l'utente e molto più semplice da mantenere rispetto a strutture di indicizzazione complesse (come B-Tree o Hash Index invertiti).

5.2.3. Algoritmo di Mapping basato su Parole Chiave

- **Algoritmo:** Implementato in `CuisineImageManager.getMacroGroupName()`. È una forma di **classificazione testuale basata su regole** con struttura a cascata di if-else.
- **Processo:**
 - **Validazione input:** Se la stringa è ***null*** o vuota, restituisce ***default.jpg***.
 - **Normalizzazione:** Converte la stringa di cucina in minuscolo (es. "Creative, Contemporary" → "creative, contemporary").
 - **Controlli sequenziali per categoria:** Valuta una sequenza di blocchi ***if*** in ordine. Ogni blocco rappresenta una categoria (es. "Italian", "French", "Japanese") e controlla se la stringa contiene almeno una delle parole chiave associate usando ***String.contains()*** con operatori OR.
 - **Early return:** Se una categoria corrisponde, restituisce immediatamente il nome dell'immagine associata (es. "italian.jpg") e termina.
 - **Fallback:** Se nessuna categoria corrisponde dopo tutti i controlli, restituisce ***default.jpg***.
- **Complessità:** $O(c \times L)$, dove:
 - **c** = numero totale di parole chiave in tutte le categorie
 - **L** = lunghezza della stringa di input
- **Giustificazione:** L'algoritmo è semplice, interpretabile e facilmente estensibile. Per un numero limitato di categorie, le performance sono adeguate. L'ordine dei controlli è importante: categorie più specifiche

dovrebbero essere controllate prima di quelle generiche (es. "Pizza" prima di "Italian").

6. Gestione dei file

6.1. File di Dati e Loro Funzione

L'applicazione utilizza 4 file principali per la persistenza dei dati:

1. **micelin_my_maps.csv**: Funge da database per il catalogo dei ristoranti.
2. **users.csv**: Funge da database per gli utenti registrati, memorizzando le loro informazioni e le password (sotto forma di hash).
3. **favorites.csv**: Funge da database per la lista di ristoranti preferiti degli utenti.
4. **reviews.csv**: Funge da database per le recensioni sui ristoranti.

6.2. Formattazione dei File

- **Formato**: CSV (Comma-Separated Values).
- **Struttura**: La prima riga è un'intestazione (header) che descrive le colonne. Ogni riga successiva è un record.
- **Delimitatore**: La virgola (,) è usata per separare i campi.
- **Parsing**: Il sistema legge i dati basandosi sulla **posizione** delle colonne, non sul nome. Questo rende il formato rigido e sensibile a modifiche nell'ordine delle colonne.

6.3. La Cartella resources e il Processo di Copia

- **Ruolo della Cartella resources**: La directory `app/src/main/resources/data` contiene le **versioni originali e di sola lettura** dei file CSV. Questi file sono impacchettati all'interno dell'applicazione.
- **Copia all'Avvio**: Al primo avvio, l'applicazione controlla se esiste una cartella di dati in una posizione scrivibile dall'utente (solitamente nella sua cartella home). Se questa cartella o i file non sono presenti, l'applicazione **copia i file originali** dalla directory `resources` a questa nuova posizione. Tutte le operazioni di lettura e scrittura successive avverranno su questa copia, lasciando intatti i file originali.

6.4. Utility per l'Aggiunta di ID

- **Funzione**: Questo strumento analizza il file e aggiunge un **identificatore numerico univoco (ID)** a ogni riga/ristorante, garantendo che ogni record abbia una chiave primaria. Questo è necessario perché i dati di origine potrebbero non avere un ID.

6.5. Modalità di Accesso ai File

L'accesso ai file segue una strategia basata su una cache in-memory per ottimizzare le prestazioni:

- **Caricamento Iniziale in Memoria:** All'avvio dell'applicazione (o meglio, alla prima richiesta di un servizio che dipende da un file), l'intero contenuto del file CSV viene letto dal disco e caricato in una lista in memoria (una cache).
- **Operazioni di Lettura:** Tutte le operazioni di lettura (ricerche, filtri, recupero di un record) vengono eseguite **esclusivamente sulla cache in memoria**. Questo approccio è estremamente rapido in quanto evita continui accessi al disco, che è un'operazione lenta.
- **Operazioni di Scrittura/Modifica:** Quando un record viene aggiunto, modificato o eliminato, l'operazione viene eseguita in due passaggi:
 1. La modifica viene prima applicata alla lista in memoria.
 2. Subito dopo, **l'intera lista aggiornata viene riscritta completamente sul file CSV** sul disco, sovrascrivendo il contenuto precedente.
- **Implicazioni:** Questa strategia garantisce che i dati siano sempre persistenti dopo una modifica e che le letture siano veloci. Tuttavia, lo svantaggio è che può essere inefficiente in termini di I/O e memoria se i file CSV diventassero estremamente grandi (milioni di righe), poiché l'intero file viene letto all'inizio e riscritto a ogni singola modifica.

7. Bibliografia

- Oracle. (s.d.). Java Platform, Standard Edition (JDK) 24 Documentation. Tratto da oracle.com
- OpenJFX. (s.d.). JavaFX 21.0.4 Documentation. Tratto da openjfx.io
- Apache Software Foundation. (s.d.). Apache Commons CSV 1.11.0. Tratto da commons.apache.org
- Microsoft Learn. (s.d.). The MVVM (Model-View-ViewModel) Pattern. Tratto da learn.microsoft.com
- Kaliski, B. (2000). PKCS #5: Password-Based Cryptography Specification Version 2.0 (RFC 2898). Internet Engineering Task Force (IETF). Recuperato da <https://tools.ietf.org/html/rfc2898>